

Capítulo 7

SQL - consultas - introdução

Comando SELECT

- ♦ Consultas em SQL são realizadas através do comando SELECT
- ♦ Sintaxe vasta – apresentação vai até o Capítulo 10
- ♦ Neste capítulo $\Rightarrow$
 - funcionalidade análoga a da álgebra relacional vista no Capítulo 3

SQL - expressões

Expressões

- ✦ Expressões são centrais no comando **SELECT**
- ✦ Há dois tipos:
 - **expressão de valor**
 - retorna um valor escalar (número, cadeia de caracteres,...)
 - **expressão de tabela**
 - retorna uma tabela

Expressão de valor

expressãoValor

- constante
- | referenciaColuna
- | expressãoValor { + | - | * | / | ** } expressãoValor
- | expressãoValor || expressãoValor
- | UPPER (expressãoValor)
- | LOWER (expressãoValor)
- | expressãoLógica
- | (expressãoValor)
- | (subConsultaEscalar)

referenciaColuna

[nomeTabela].nomeColuna

Expressão de valor - exemplos

```
/* === constantes === */
125                /* número inteiro */
54.454            /* número decimal */
'Rio'              /* cadeia de caracteres */
'2000-02-30'       /* data */
NULL               /* vazio */

/* === referências à coluna === */
nome_peca
peca.nome_peca

/* === expressões numéricas === */
5 * ( 12 + peso_peca )
peso_peca - 12.5

/* === expressões sobre
                               cadeias de caracteres === */
cidade || ' - RS' /* concatenação */
/* funções */
UPPER(cidade) /* converte para maiúsculas */
LOWER(cidade) /* converte para minúsculas */
```

Expressão lógica

expressãoLógica

expressãoValor

constante
| referenciaColuna
| expressãoValor { + | - | * | /
| expressãoValor || expressãoValor
| UPPER (expressãoValor)
| LOWER (expressãoValor)
| expressãoLógica
| (expressãoValor)
| (subConsultaEscalar)

TRUE
| FALSE
| UNKNOWN
| expressãoValor { = | <> | != | < | <= | > | >= | LIKE }
| expressãoValor
| expressãoValor IS NULL
| expressãoValor IS NOT NULL
| NOT expressãoLógica
| expressãoLógica OR expressãoLógica
| expressãoLógica AND expressãoLógica
| expressãoLógicaExists
| expressãoLógicaIN

Expressão lógica

- exemplos

```
/* === constantes === */
```

```
TRUE
```

```
FALSE
```

```
UNKNOWN
```

```
/* === comparações de valores === */
```

```
5 = peso_pecas
```

```
peso_pecas <> 12.5
```

```
peso_pecas != 12.5
```

```
cidade_pecas LIKE 'Ri%'
```

```
/* === testes por vazio === */
```

```
cidade_pecas IS NULL
```

```
cidade_pecas IS NOT NULL
```

```
/* === conectores lógicos === */
```

```
cidade_pecas != 'Rio' AND peso_pecas = 10
```

```
NOT cidade_pecas = 'Rio'
```


Comando SELECT - sintaxe

comandoSelect

SELECT

[ALL | DISTINCT] { * | colunaSelect [, ...] }

FROM tabelaSelect [, ...]

[WHERE expressãoLógica]

[GROUP BY expressãoValor [, ...]]

[HAVING expressãoLógica]

[{ UNION | INTERSECT | EXCEPT } [ALL]

comandoSelect

]

[ORDER BY { expressãoValor [ASC | DESC] } [, ...]]

SELECT

seleção, projeção, produto cartesiano

Seleção, projeção, produto cartesiano

- ♦ Primeira versão mostrada implementa seleção, projeção, produto cartesiano

comandoSelect

SELECT

[ALL | DISTINCT] { * | colunaSelect [, ...] }

FROM tabelaSelect [, ...]

[WHERE expressãoLógica]

[GROUP BY expressãoValor [, ...]]

[HAVING expressãoLógica]

[{ UNION | INTERSECT | EXCEPT } [ALL]

comandoSelect

]

[ORDER BY { expressãoValor [ASC | DESC] } [, ...]]

para começar,
consideramos apenas
estas três cláusulas

Seleção, projeção, produto cartesiano

- ♦ Primeira versão mostrada implementa seleção, projeção, produto cartesiano

comandoSelect

```
SELECT
  [ ALL | DISTINCT ] { * | colunaSelect [ , ... ] }
FROM tabelaSelect [ , ... ]
[ WHERE expressãoLógica ]
[ GROUP BY expressãoValor [ , ... ] ]
[ HAVING expressãoLógica ]
[ { UNION | INTERSECT | EXCEPT } [ ALL ]
  comandoSelect
]
[ ORDER BY { expressãoValor [ ASC | DESC ] } [ , ... ] ]
```

colunaSelect é uma referência a coluna

Seleção, projeção, produto cartesiano

- ♦ Primeira versão mostrada implementa seleção, projeção, produto cartesiano

comandoSelect

SELECT

```
[ ALL | DISTINCT ] { * | colunaSelect [
FROM tabelaSelect [ , ... ]
[ WHERE expressãoLógica ]
[ GROUP BY expressãoValor [ , ... ] ]
[ HAVING expressãoLógica ]
[ { UNION | INTERSECT | EXCEPT } [ ALL ]
  comandoSelect
]
[ ORDER BY { expressãoValor [ ASC | DESC ] } [ , ... ] ]
```

tabelaSelect é um
nome de tabela

Seleção, projeção, produto cartesiano

- ♦ Primeira versão mostrada implementa seleção, projeção, produto cartesiano

comandoSelect

```
SELECT
  [ ALL | DISTINCT ] { * | colunaSelect [ , ... ] }
FROM tabelaSelect [ , ... ]
[ WHERE expressãoLógica ]
[ GROUP BY expressãoValor [ , ... ] ]
[ HAVING expressãoLógica ]
[ { UNION | INTERSECT | EXCEPT } [ ALL ]
  comandoSelect
]
[ ORDER BY { expressãoValor [ ASC | DESC ] } [ , ... ] ]
```

expressãoLógica envolve
apenas constantes e
nomes de campos

Avaliação do comando **SELECT** (primeira versão)

- ♦ Passo 1: **produto cartesiano** das tabelas especificadas pelos termos **tabelaSelect**.
- ♦ Passo 2: **seleção** usando a condição especificada em **condiçãoSelect**
- ♦ Passo 3: **projeção** pelas colunas especificadas pelos termos **colunaSelect**
(se for **SELECT ***, todas colunas vão para o resultado)

Exemplo 7.1

(BD Embarques)

Obter a tabela de peças

Exemplo 7.1

(BD Embarques)

Obter a tabela de peças

```
SELECT *  
FROM peca
```

Exemplo 7.2

(BD Embarques)

Obter as peças que são da cidade 'Rio'

Exemplo 7.2

(BD Embarques)

Obter as peças que são da cidade 'Rio'

```
SELECT *  
FROM   peca  
WHERE  cidade_peca = 'Rio'
```

Exemplo 7.3

(BD Embarques)

Obter o código, o nome e o peso de cada peça cuja cidade é 'Rio'

Exemplo 7.3

(BD Embarques)

Obter o código, o nome e o peso de cada peça cuja cidade é 'Rio'

```
SELECT
    cod_peca ,
    nome_peca ,
    peso_peca
FROM   peca
WHERE  cidade_peca = 'Rio'
```

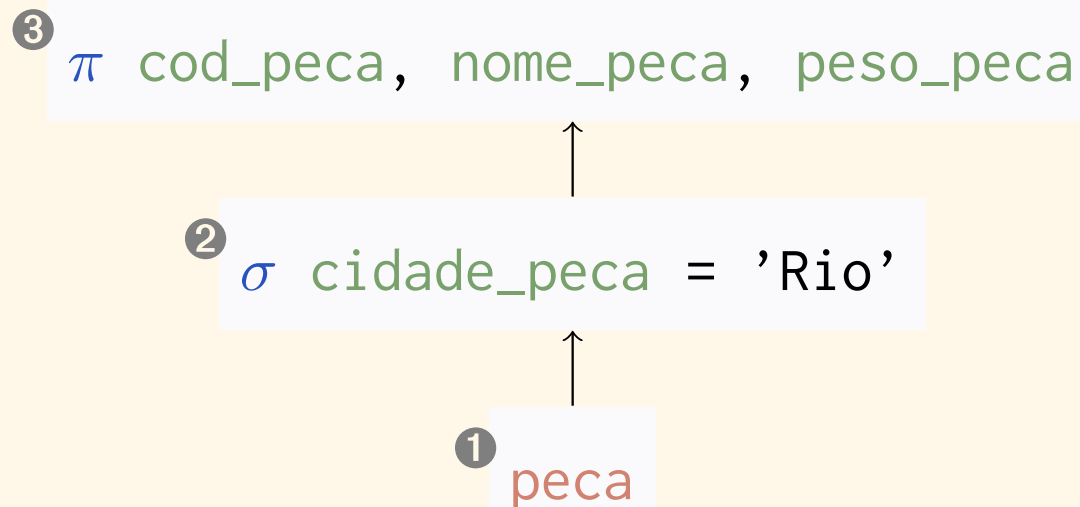
Exemplo 7.3

(equivalência em álgebra – representação em árvore)

(BD Embarques)

Obter o código, o nome e o peso de cada peça cuja cidade é 'Rio'

```
SELECT  
    cod_peca ,  
    nome_peca ,  
    peso_peca  
FROM   peca  
WHERE  cidade_peca = 'Rio'
```



Exemplo 7.3

(equivalência em álgebra – representação textual)

(BD Embarques)

Obter o código, o nome e o peso de cada peça cuja cidade é 'Rio'

```
SELECT
    cod_pecas ,
    nome_pecas ,
    peso_pecas
FROM   pecas
WHERE  cidade_pecas = 'Rio'
```

```
 $\pi$  cod_pecas ,
    nome_pecas ,
    peso_pecas
    (  $\sigma$  cidade_pecas = 'Rio'
      (pecas)
    )
```

Exemplo 7.4

(BD Embarques)

Obter as peças que são da cidade denominada 'Rio' e cujo peso excede 15

Exemplo 7.4

(BD Embarques)

Obter as peças que são da cidade denominada 'Rio' e cujo peso excede 15

```
SELECT *  
FROM   peca  
WHERE  ( cidade_peca = 'Rio' AND  
        peso_peca > 15  
        )
```

Exemplo 7.5

(BD Embarques)

Para cada embarque de uma peça denominada 'Parafuso', obter o código da peça, o código do fornecedor e a data do embarque

Exemplo 7.5

(BD Embarques)

Para cada embarque de uma peça denominada 'Parafuso', obter o código da peça, o código do fornecedor e a data do embarque

```
SELECT
    peca.cod_peca ,
    cod_fornec ,
    data_embarq
FROM    peca ,
        embarque
WHERE   peca.cod_peca = embarque.cod_peca
AND     peca.nome_peca = 'Parafuso '
```

Exemplo 7.6

(BD Embarques)

Para cada embarque de uma peça denominada 'Parafuso', obter o código da peça, o código do fornecedor, o nome do fornecedor, e a data do embarque

Exemplo 7.6

(BD Embarques)

Para cada embarque de uma peça denominada 'Parafuso', obter o código da peça, o código do fornecedor, o nome do fornecedor, e a data do embarque

```
SELECT
    peca.cod_pecas ,
    embarque.cod_fornec ,
    nome_fornec ,
    data_embarq
FROM    peca ,
        embarque ,
        fornecedor
WHERE   peca.cod_pecas = embarque.cod_pecas
        AND fornecedor.cod_fornec =
            embarque.cod_fornec
        AND nome_pecas = 'Parafuso'
```

Exemplo 7.6

(BD Embarques)

Para cada embarque de uma peça denominada 'Parafuso', obter o código da peça, o código do fornecedor, o nome do fornecedor, e a data do embarque

```
SELECT
    peca.cod_peca,
    embarque.cod_fornec,
    nome_fornec,
    data_embarq
FROM peca,
     embarque,
     fornecedor
WHERE peca.cod_peca = embarque.cod_peca
      AND fornecedor.cod_fornec =
          embarque.cod_fornec
      AND nome_peca = 'Parafuso'
```

nome da coluna qualificado
com nome da tabela

-
necessário, pois `cod_peca`
aparece tanto em `peca`
quanto em `fornecedor`

Cadeias de caracteres

- ✦ Comparação de cadeias é estrita:
 - maiúsculas diferente de minúsculas, ...
- ✦ Funções como **UPPER** e **LOWER** podem ser usadas

Exemplo 7.7

(BD Embarques)

*Obter os dados dos fornecedores cuja cidade é diferente de 'Porto Alegre',
incluindo daqueles que não têm cidade informada*

Exemplo 7.7

(BD Embarques)

Obter os dados dos fornecedores cuja cidade é diferente de 'Porto Alegre', incluindo daqueles que não têm cidade informada

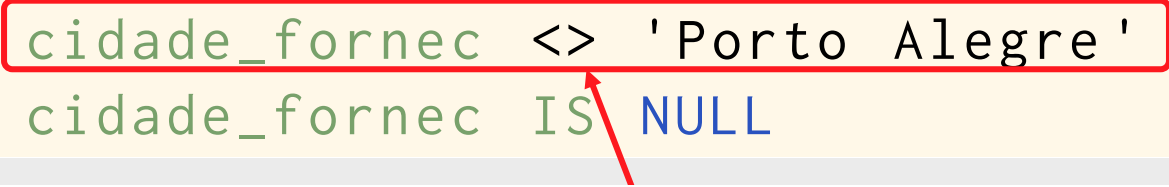
```
SELECT *  
FROM   fornecedor  
WHERE  cidade_fornec <> 'Porto Alegre'  
       OR cidade_fornec IS NULL
```

Exemplo 7.7

(BD Embarques)

Obter os dados dos fornecedores cuja cidade é diferente de 'Porto Alegre', incluindo daqueles que não têm cidade informada

```
SELECT *  
FROM fornecedor  
WHERE cidade_fornec <> 'Porto Alegre'  
OR cidade_fornec IS NULL
```



- esta comparação é estrita
- p.ex., 'porto alegre' não vai para o resultado
- se isso não for desejável, usar função **UPPER** ou **LOWER**

Exemplo 7.7

(BD Embarques)

Obter os dados dos fornecedores cuja cidade é diferente de 'Porto Alegre', incluindo daqueles que não têm cidade informada

```
SELECT *  
FROM   fornecedor  
WHERE  cidade_fornec <> 'Porto Alegre'  
       OR cidade_fornec IS NULL
```



- recordar capítulo 3
- SQL usa a lógica de três valores

Exemplo 7.8

(BD Embarques)

Obter as peças que são da cidade denominada 'Rio'. A palavra 'Rio' pode estar escrita em qualquer combinação de letras maiúsculas e minúsculas

Exemplo 7.8

(BD Embarques)

Obter as peças que são da cidade denominada 'Rio'. A palavra 'Rio' pode estar escrita em qualquer combinação de letras maiúsculas e minúsculas

```
SELECT *  
FROM   peca  
WHERE  UPPER(cidade_peca) = 'RIO'
```

ou

```
SELECT *  
FROM   peca  
WHERE  LOWER(cidade_peca) = 'rio'
```

Casamento de padrões

cadeia LIKE padrão

- ♦ Operador LIKE compara **cadeia** com o **padrão**
- ♦ **padrão:**
 - sublinhado () serve como coringa para um caractere qualquer
 - símbolo de porcentagem (%) serve como coringa para uma cadeia qualquer

LIKE - exemplos

✦ Comparações que resultam em verdadeiro

- 'Pedro' LIKE 'Pedro'
- 'Pedro' LIKE '_edro'
- 'Pedro' LIKE 'P%o'

✦ Comparações que resultam em falso

- 'Pedro' LIKE 'Ped'
- 'Pedro' LIKE 'P_o'

Exemplo 7.9

(BD Embarques)

Obter o código e o nome de todos fornecedores cujo nome começa com a letra 'S' maiúscula

Exemplo 7.9

(BD Embarques)

Obter o código e o nome de todos fornecedores cujo nome começa com a letra 'S' maiúscula

```
SELECT
    cod_fornec ,
    nome_fornec
FROM   fornecedor
WHERE  nome_fornec LIKE 'S%'
```

Tratamento de duplicatas

- ✦ Em SQL (ao contrário da abordagem relacional teórica) $\Rightarrow$
 - o resultado de uma consulta pode conter duplicatas
- ✦ Tratamento especificado pelas palavras-chave **ALL** e **DISTINCT**

Tratamento de duplicatas

comandoSelect

```
SELECT
  [ ALL | DISTINCT ] { * | colunaSelect [ , ... ] }
FROM tabelaSelect [ , ... ]
[ WHERE expressãoLógica ]
[ GROUP BY expressãoValor [ , ... ] ]
[ HAVING expressãoLógica ]
[ { UNION | INTERSECT | EXCEPT } [ ALL ]
  comandoSelect
]
[ ORDER BY { expressãoValor [ ASC | DESC ] } [ , ... ] ]
```

DISTINCT
especifica que o
resultado é um
conjunto (duplicatas
são eliminadas)

Tratamento de duplicatas

comandoSelect

```
SELECT
  [ ALL | DISTINCT ] { * | colunaSelect [ , ... ] }
  FROM tabelaSelect [ , ... ]
  [ WHERE expressãoLógica ]
  [ GROUP BY expressãoValor [ , ... ] ]
  [ HAVING expressãoLógica ]
  [ { UNION | INTERSECT | EXCEPT } [ ALL ]
    comandoSelect
  ]
  [ ORDER BY { expressãoValor [ ASC | DESC ] } [ , ... ] ]
```

ALL (opção *default*)
especifica que o
resultado é um multi-
conjunto (pode conter
linhas duplicadas)

Exemplo 7.10

(BD Embarques)

Obter as diferentes cidades em que há fornecedores

Exemplo 7.10

(BD Embarques)

Obter as diferentes cidades em que há fornecedores

```
SELECT DISTINCT  
    cidade_fornec  
FROM fornecedor
```

Exemplo 7.10 - resultado

(BD Embarques)

Obter as diferentes cidades em que há fornecedores

```
SELECT DISTINCT  
    cidade_fornec  
FROM fornecedor
```

<code>cidade_fornec</code>
Porto Alegre
Curitiba
Rio
<N>

Exemplo 7.11

(BD Embarques)

Obter as diferentes cidades em que há fornecedores. Caso algum fornecedor não tenha cidade informada, o vazio não deve constar do resultado.

Exemplo 7.11

(BD Embarques)

Obter as diferentes cidades em que há fornecedores. Caso algum fornecedor não tenha cidade informada, o vazio não deve constar do resultado.

```
SELECT DISTINCT  
    cidade_fornec  
FROM    fornecedor  
WHERE   cidade_fornec IS NOT NULL
```

Exemplo 7.12

(BD Embarques)

Obter as cidades em que há fornecedores. A cidade deve aparecer no resultado tantas vezes quantos fornecedores nela existirem.

Exemplo 7.12

(BD Embarques)

Obter as cidades em que há fornecedores. A cidade deve aparecer no resultado tantas vezes quantos fornecedores nela existirem.

```
SELECT ALL  
       cidade_fornec  
FROM   fornecedor
```

Exemplo 7.12 - resultado

(BD Embarques)

Obter as cidades em que há fornecedores. A cidade deve aparecer no resultado tantas vezes quantos fornecedores nela existirem.

```
SELECT ALL  
       cidade_fornec  
FROM   fornecedor
```

cidade_fornec
Porto Alegre
Porto Alegre
Curitiba
Rio
<N>
Rio

Exemplo 7.13

(BD Embarques)

Escreva uma consulta em SQL, que, para cada professor, obtenha seu nome seguido do nome de seu departamento.

Exemplo 7.13

(BD Embarques)

Escreva uma consulta em SQL, que, para cada professor, obtenha seu nome seguido do nome de seu departamento.

```
SELECT ALL
    professor.nome_prof ,
    depto.nome_depto
FROM    professor ,
        depto
WHERE   depto.cod_depto =
        professor.cod_depto
```

Exemplo 7.13

(BD Embarques)

Escreva uma consulta em SQL, que, para cada professor, obtenha seu nome seguido do nome de seu departamento.

```
SELECT ALL  
    professor.nome_pro  
    depto.nome_depto  
FROM professor ,  
depto  
WHERE depto.cod_depto =  
        professor.cod_depto
```

aqui **DISTINCT** pode levar a resultado incorreto:
caso de professores homônimos
em departamentos homônimos

Exemplo 7.14

(BD Embarques)

Obter o código, o nome e o peso de cada peça para a qual existe ao menos um embarque do fornecedor de código 'F1'

Exemplo 7.14

(BD Embarques)

Obter o código, o nome e o peso de cada peça para a qual existe ao menos um embarque do fornecedor de código 'F1'

```
SELECT DISTINCT
    peça.cod_peça ,
    nome_peça ,
    peso_peça
FROM   peça ,
       embarque
WHERE  peça.cod_peça = embarque.cod_peça
      AND embarque.cod_fornec = 'F1'
```

DISTINCT é necessário:
caso contrário uma peça com
múltiplos embarques aparece repetida

Eliminação de duplicatas e desempenho

- ✦ Eliminação de duplicatas pode ser **custosa**:
 - custo de ordenação em memória externa ou equivalente
- ✦ Só usar **DISTINCT** se **necessário**
- ✦ Usar quando:
 - sem **DISTINCT** podem haver duplicatas no resultado
 - a **eliminação** destas duplicatas é **requerida** pela aplicação

SELECT

operações de conjunto

SELECT - operações de conjunto

comandoSelect

SELECT

[ALL | DISTINCT] { * | colunaSelect [, ...] }

FROM tabelaSelect [, ...]

[WHERE expressãoLógica]

[GROUP BY expressãoValor [, ...]]

[HAVING expressãoLógica]

[{ UNION | INTERSECT | EXCEPT } [ALL]

comandoSelect

]

[ORDER BY { expressãoValor [ASC | DESC] } [, ...]]

Compatibilidade para operação de conjunto (regras iguais às da álgebra)

- ✦ Nem todo par de tabelas pode ser usado como operando de uma operação de conjuntos
- ✦ Requisitos para que duas tabelas sejam **compatíveis para operação de conjunto** $\Rightarrow$
 - devem ter o **mesmo número de colunas**
 - **domínio** da i -ésima coluna da tabela₁ deve ser igual ao domínio da i -ésima coluna da tabela₂

Operações de conjunto - resultado

♦ União (**UNION**) $\Rightarrow$

- conjunto das linhas que aparecem em uma **ou** outra tabela

♦ Interseção (**INTERSECT**) $\Rightarrow$

- conjunto de linhas que aparecem **em ambas** as tabelas

♦ Diferença (**MINUS**) $\Rightarrow$

- conjunto das linhas que aparecem na primeira tabela e não aparecem na segunda tabela

Exemplo 7.15

(BD Embarques)

Obter as cidades em que há tanto uma peça quanto um fornecedor

Exemplo 7.15

(BD Embarques)

Obter as cidades em que há tanto uma peça quanto um fornecedor

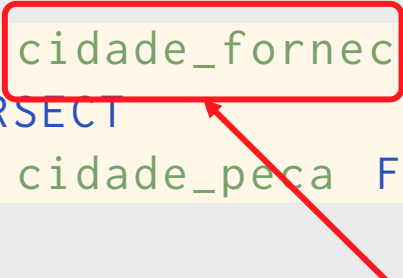
```
SELECT cidade_fornec FROM fornecedor  
INTERSECT  
SELECT cidade_peca FROM peca
```


Exemplo 7.15

(BD Embarques)

Obter as cidades em que há tanto uma peça quanto um fornecedor

```
SELECT cidade_fornec FROM fornecedor  
INTERSECT  
SELECT cidade_peca FROM peca
```



convenção $\Rightarrow$
tabela resultante têm os nomes de
coluna da primeira tabela

SELECT - operações de conjunto

tratamento de duplicatas

comandoSelect

SELECT

[ALL | DISTINCT] { * | colunaSe

FROM tabelaSelect [, ...]

[WHERE expressãoLógica]

[GROUP BY expressãoValor [, ...]]

[HAVING expressãoLógica]

[{ UNION | INTERSECT | EXCEPT }

comandoSelect

]

[ORDER BY { expressãoValor [ASC | DESC] } [, ...]]

se esta opção for omitida
duplicatas são eliminadas do resultado

[ALL]

Exemplo 7.16

(BD Embarques)

*Obter as cidades nas quais se encontra uma peça ou um fornecedor.
O resultado pode conter duplicatas.*

Exemplo 7.16

(BD Embarques)

*Obter as cidades nas quais se encontra uma peça ou um fornecedor.
O resultado pode conter duplicatas.*

```
SELECT cidade_fornec FROM fornecedor  
UNION ALL  
SELECT cidade_peca FROM peca
```

Tabelas e colunas no comando **SELECT**

Tabelas e colunas no SELECT - alternativas

- ♦ atribuir **nomes** a **colunas** do resultado
- ♦ definir **expressões** que computam os valores de **colunas** do resultado
- ♦ atribuir **nomes** a **tabelas** que aparecem na consulta
- ♦ definir **expressões** que computam **tabelas** a utilizar na consulta

Definição de coluna - sintaxe

colunaSelect

```
{ referenciaColuna | expressãoValor }  
  [ [ AS ] nomeColunaAlias ]
```

Definição de tabela - sintaxe

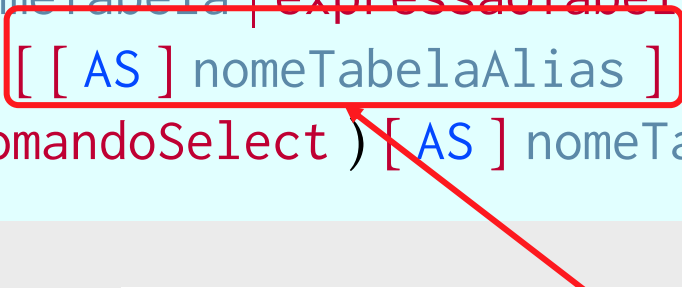
tabelaSelect

```
{ nomeTabela | expressãoTabela }  
  [ [ AS ] nomeTabelaAlias ]  
| ( comandoSelect ) [ AS ] nomeTabelaAlias
```


Nome (aliás) de tabela

tabelaSelect

```
{ nomeTabela | expressãoTabela }  
[ [ AS ] nomeTabelaAlias ]  
| ( comandoSelect ) [ AS ] nomeTabelaAlias
```



nome (aliás) dado a uma tabela dentro de uma consulta
-
palavra-chave **AS** é opcional

Exemplo 7.17

(BD Embarques)

*Obter o código, o nome e o peso de cada peça
para a qual exista ao menos um embarque do fornecedor de código 'F1'*

Exemplo 7.17

(BD Embarques)

Obter o código, o nome e o peso de cada peça para a qual exista ao menos um embarque do fornecedor de código 'F1'

```
SELECT DISTINCT
    p.cod_peca ,
    nome_peca ,
    peso_peca
FROM   peca AS p,
       embarque e
WHERE  ( p.cod_peca = e.cod_peca AND
        e.cod_fornec = 'F1'
      )
```

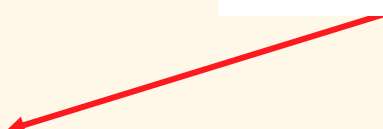
Exemplo 7.17

(BD Embarques)

Obter o código, o nome e o peso de cada peça para a qual exista ao menos um embarque do fornecedor de código 'F1'

```
SELECT DISTINCT
    p.cod_peca ,
    nome_peca ,
    peso_peca
FROM   peca AS p,
       embarque e
WHERE  ( p.cod_peca = e.cod_peca AND
        e.cod_fornec = 'F1'
      )
```

nome (aliás) usado para referir à tabela **peca** dentro da consulta



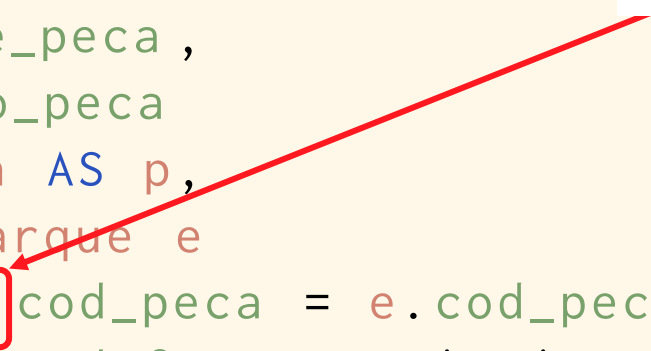
Exemplo 7.17

(BD Embarques)

Obter o código, o nome e o peso de cada peça para a qual exista ao menos um embarque do fornecedor de código 'F1'

```
SELECT DISTINCT
    p.cod_peca ,
    nome_peca ,
    peso_peca
FROM   peca AS p,
       embarque e
WHERE  ( p.cod_peca = e.cod_peca AND
        e.cod_fornec = 'F1'
      )
```

referência à tabela **peca**
através de seu aliás



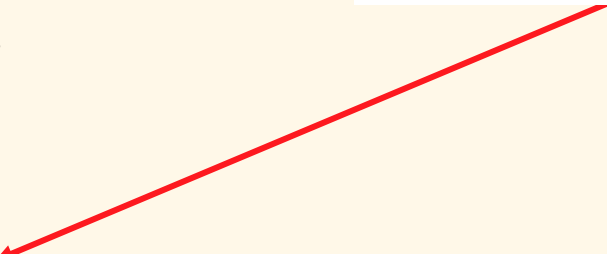
Exemplo 7.17

(BD Embarques)

Obter o código, o nome e o peso de cada peça para a qual exista ao menos um embarque do fornecedor de código 'F1'

```
SELECT DISTINCT
    p.cod_peca ,
    nome_peca ,
    peso_peca
FROM   peca AS p,
       embarque e
WHERE  ( p.cod_peca = e.cod_peca AND
        e.cod_fornec = 'F1'
      )
```

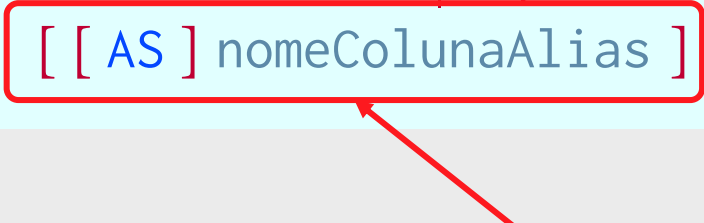
palavra chave **AS** é opcional



Renomeação de coluna

colunaSelect

```
{ referenciaColuna | expressãoValor }  
[ [ AS ] nomeColunaAlias ]
```



nome (aliás) dado a uma coluna do resultado
-
palavra-chave **AS** é opcional

Exemplo 7.18

(BD Embarques)

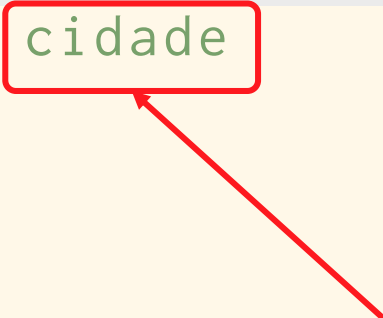
*Obter as cidades em que há tanto uma peça quanto um fornecedor.
A coluna do resultado deve ter o nome de cidade.*

Exemplo 7.18

(BD Embarques)

*Obter as cidades em que há tanto uma peça quanto um fornecedor.
A coluna do resultado deve ter o nome de cidade.*

```
SELECT cidade_fornec AS cidade
FROM fornecedor
INTERSECT
SELECT cidade_peca
FROM peca
```



nome (aliás) da coluna no resultado

Expressão define coluna do resultado

colunaSelect

```
{ referenciaColuna | expressãoValor }  
  [ [ AS ] nomeColunaAlias ]
```

valor de uma coluna pode ser definido
por uma expressão de valor

Exemplo 7.19

(BD Embarques)

Para cada peça, obter seu código, seu nome, e seu peso em libras.

O peso das peças no banco de dados encontra-se em quilogramas e o fator de conversão para libras é 2,2046. A coluna contendo o peso da peça em libras deve ter o nome `peso_peca_libras`.

Exemplo 7.19

(BD Embarques)

Para cada peça, obter seu código, seu nome, e seu peso em libras.

O peso das peças no banco de dados encontra-se em quilogramas e o fator de conversão para libras é 2,2046. A coluna contendo o peso da peça em libras deve ter o nome `peso_peca_libras`.

```
SELECT
    cod_peca ,
    nome_peca ,
    peso_peca * 2.2046
        AS peso_libras_peca
FROM peca
```

expressão que define o valor da
terceira coluna do resultado

Exemplo 7.19

(BD Embarques)

Para cada peça, obter seu código, seu nome, e seu peso em libras.

O peso das peças no banco de dados encontra-se em quilogramas e o fator de conversão para libras é 2,2046. A coluna contendo o peso da peça em libras deve ter o nome `peso_peca_libras`.

```
SELECT
```

```
    cod_peca ,
```

```
    nome_peca ,
```

```
    peso_peca * 2.2046
```

```
        AS peso_libras_peca
```

```
FROM peca
```

nome da terceira coluna do resultado



Combinando várias linhas de uma tabela

- ✦ em consulta que combina várias linhas de uma **mesma** tabela
 - é necessário um mecanismo que permita distinguir entre estas várias linhas
- ✦ na álgebra relacional usa-se a **renomeação**
- ✦ em SQL é possível definir aliás para tabelas

Exemplo 7.20

Obter uma tabela contendo uma linha para cada empregado que possui um chefe. Esta linha deve conter o nome do empregado, seguido do nome de seu chefe.

empregado

codigo_emp	nome	cod_emp_chefe
10	Pereira	<N>
21	Tavares	10
30	Santos	10
55	Almeida	21

- chefes são empregados
- constam da mesma tabela

Exemplo 7.20

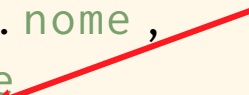
- ✦ Solução envolve concatenar duas linhas de **empregado**:
 - a linha do próprio empregado
 - a linha do chefe do empregado
- ✦ Como distinguir as colunas destas duas linhas?
 - Solução: **renomear** uma delas

Exemplo 7.20

Obter uma tabela contendo uma linha para cada empregado que possui um chefe. Esta linha deve conter o nome do empregado, seguido do nome de seu chefe.

```
SELECT
    empregado.nome,
    chefe.nome
FROM empregado,
    empregado AS chefe
WHERE empregado.cod_emp_chefe =
    chefe.codigo_emp
```

tabela **empregado**
fornece as linhas
dos empregados

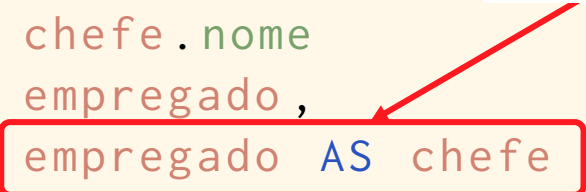


Exemplo 7.20

Obter uma tabela contendo uma linha para cada empregado que possui um chefe. Esta linha deve conter o nome do empregado, seguido do nome de seu chefe.

```
SELECT
    empregado.nome ,
    chefe.nome
FROM
    empregado ,
    empregado AS chefe
WHERE
    empregado.cod_emp_chefe =
        chefe.codigo_emp
```

tabela **empregado** renomeada como **chefe** fornece as linhas dos chefes



Exemplo 7.20 (resultado)

empregado

codigo_emp	nome	cod_emp_chefe
10	Pereira	<N>
21	Tavares	10
30	Santos	10
55	Almeida	21



tabela resultado

empregado. nome	emp_chefe. nome
Tavares	Pereira
Santos	Pereira
Almeida	Tavares

Subconsulta

Subconsulta

subconsulta

- define uma tabela usada como operando em uma consulta mais abrangente

- tabela deve ser nomeada através de **AS**

tabelaSelect

```
{ nomeTabela | expressãoTabela }  
[ [ AS ] nomeTabelaAlias ]  
| ( comandoSelect ) [ AS ] nomeTabelaAlias
```

Exemplo 7.2I

(BD Embarques)

Para cada embarque de uma peça denominada 'Parafuso', obter o código da peça, o código do fornecedor, o nome do fornecedor e a data do embarque

Exemplo 7.2I

```
SELECT
    parafuso.cod_pecas ,
    embarque.cod_fornec ,
    nome_fornec ,
    data_embarq
FROM ( SELECT cod_pecas
      FROM pecas
      WHERE nome_pecas = 'Parafuso'
    ) AS parafuso ,
    embarque ,
    fornecedor
WHERE ( parafuso.cod_pecas =
        embarque.cod_pecas AND
        fornecedor.cod_fornec =
        embarque.cod_fornec
    )
```

Exemplo 7.21

```
SELECT
    parafuso.cod_peca ,
    embarque.cod_fornec ,
    nome_fornec ,
    data_embarq
FROM ( SELECT cod_peca
      FROM peca
      WHERE nome_peca = 'Parafuso'
    ) AS parafuso ,
    embarque ,
    fornecedor
WHERE ( parafuso.cod_peca =
        embarque.cod_peca AND
        fornecedor.cod_fornec =
        embarque.cod_fornec
    )
```

o resultado desta subconsulta
é usado como operando da
consulta mais abrangente

Exemplo 7.2I

```
SELECT
    parafuso.cod_pecas ,
    embarque.cod_fornec ,
    nome_fornec ,
    data_embarq
FROM ( SELECT cod_pecas
      FROM peca
      WHERE nome_pecas = 'Parafuso'
    ) AS parafuso,
    embarque ,
    fornecedor
WHERE ( parafuso.cod_pecas =
        embarque.cod_pecas AND
        fornecedor.cod_fornec =
        embarque.cod_fornec
    )
```

nome dado ao resultado
da subconsulta



Exemplo 7.21

```
SELECT
    parafuso.cod_pecas ,
    embarque.cod_fornec ,
    nome_fornec ,
    data_embarq
FROM ( SELECT cod_pecas
      FROM peca
      WHERE nome_pecas = 'Parafuso'
    ) AS parafuso ,
    embarque ,
    fornecedor
WHERE ( parafuso.cod_pecas =
        embarque.cod_pecas AND
        fornecedor.cod_fornec =
        embarque.cod_fornec
    )
```

na consulta mais abrangente,
resultado da subconsulta é
usado como se fosse uma
tabela do BD

Subconsulta escalar

Subconsulta escalar em expressão de valor

expressãoValor

```
constante  
| referenciaColuna  
| expressãoValor { + | - | * | / | ** }  
| expressãoValor || expressãoValor  
| UPPER ( expressãoValor )  
| LOWER ( expressãoValor )  
| expressãoLógica  
| ( expressãoValor )  
| ( subConsultaEscalar )
```



subconsulta que resulta em exatamente um linha contendo apenas um campo é chamada de subconsulta **escalar**

-
pode ser usada como um valor dentro de qualquer expressão

-
sempre aparece entre parênteses

Exemplo 7.22

(BD Embarques)

Para cada professor, obter seu código, seu nome e o nome de seu departamento

Exemplo 7.22

(BD Embarques)

Para cada professor, obter seu código, seu nome e o nome de seu departamento

```
SELECT
    cod_prof ,
    nome_prof ,
    ( SELECT nome_depto
      FROM   depto
      WHERE  cod_depto =
              professor.cod_depto
    )
FROM professor
```

Exemplo 7.22

subconsulta define o valor de uma coluna
-
deve retornar exatamente um valor

```
SELECT
  cod_prof ,
  nome_prof ,
  ( SELECT nome_depto
    FROM   depto
    WHERE  cod_depto =
           professor.cod_depto
  )
FROM professor
```